

使用基準設定檔提升應用程式效能

來源：<https://codelabs.developers.google.com/android-baseline-profiles-improve?hl=zh-tw>

步驟數：11

在本程式碼研究室中，將說明如何使用基準設定檔加快應用程式啟動時間和影格時間。

目錄

1. [1. 事前準備](#)
2. [2. 開始設定](#)
3. [3. 什麼是基準設定檔](#)
4. [4. 設定基準設定檔產生器模組](#)
5. [5. 撰寫基準設定檔產生器](#)
6. [6. 備妥用來執行產生器的裝置](#)
7. [7. 產生基準設定檔](#)
8. [8. \(選用\) 自訂基準設定檔的產生方式](#)
9. [9. 驗證啟動效能提升幅度](#)
10. [10. \(選用\) 驗證捲動效能提升幅度](#)
11. [11. 恭喜](#)

1. 事前準備

在本程式碼研究室中，您將學習如何產生基準設定檔來最佳化應用程式效能，並瞭解如何在使用基準設定檔後確認效能提升幅度。

軟硬體需求

- [Android Studio Hedgehog \(2023.1.1\)](#) 以上版本
- Android Gradle 外掛程式 8.0 以上版本
- [Jetpack Macrobenchmark 基礎知識](#)
- 搭載 Android 7 (API 級別 24) 以上版本的實體 Android 裝置

注意：如果沒有實體裝置，可以使用 Android 模擬器進行本程式碼研究室課程。不過請注意，由於模擬器和使用的 OS 共用資源，因此效能和基準測試數據無法代表實體裝置結果。

課程步驟

- 設定專案，使用基準設定檔產生器。
- 產生基準設定檔，最佳化應用程式啟動和捲動效能。
- 運用 Jetpack Macrobenchmark 程式庫確認效能提升結果。

課程內容

- 瞭解基準設定檔，以及如何用來提升應用程式效能。
 - 如何產生基準設定檔。
 - 基準設定檔的效能提升幅度。
-

步驟 2 · 約 3 分鐘

2. 開始設定

若要開始，請用以下指令從指令列複製 GitHub 存放區：

```
$ git clone https://github.com/android/codelab-android-performance.git
```

本專案是在多個 Git 分支版本中建構而成：

- **main**：本專案的範例程式碼，您需要加以修改，才能完成這個程式碼研究室。
- **end**：內含本程式碼研究室的解決方案。

或者，您也可以下載兩個 ZIP 檔案：

- [範例](#)
- [解決方案](#)

在 Android Studio 中開啟專案

1. 在「Welcome to Android Studio」視窗中，選取「



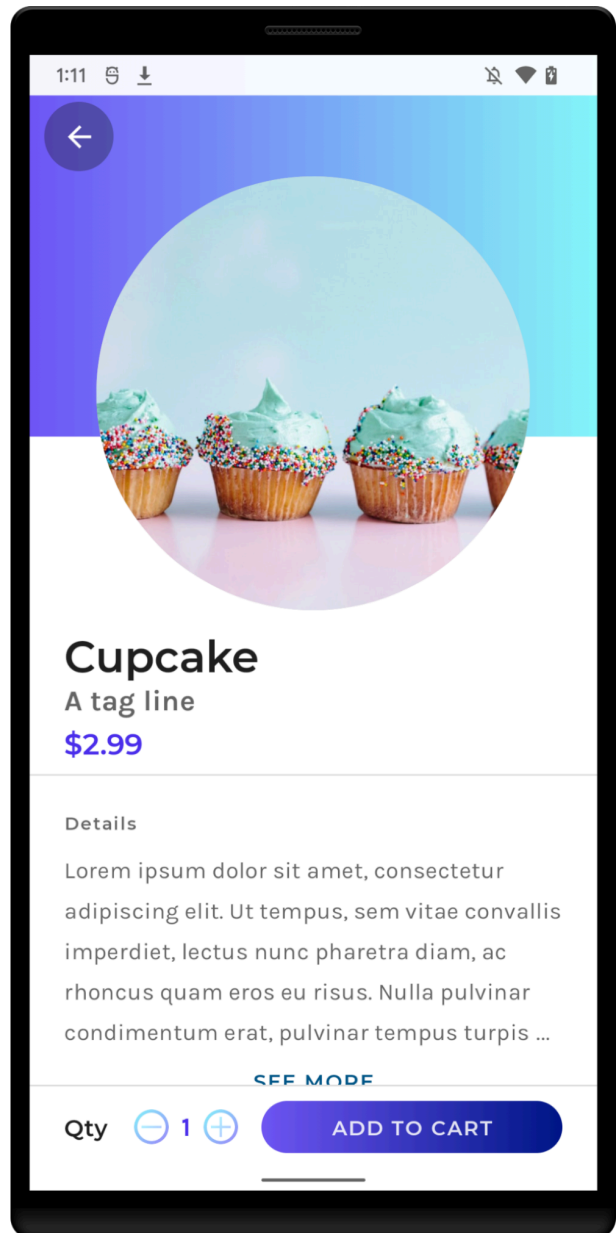
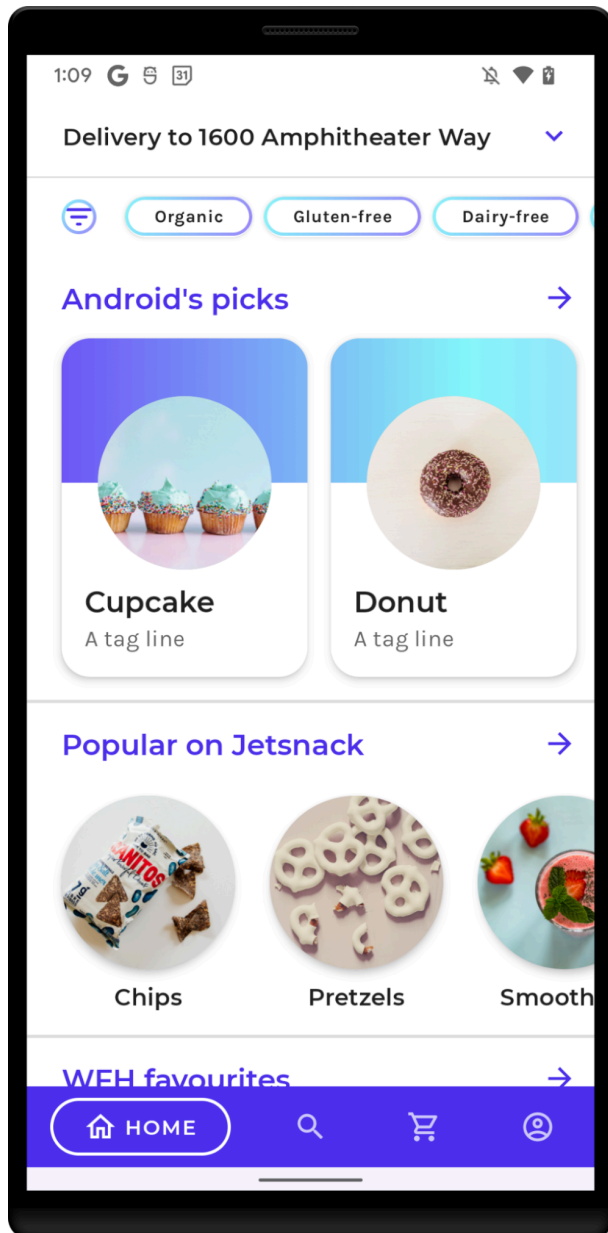
Open an Existing Project」。

2. 選取資料夾 `[Download Location]/codelab-android-performance/baseline-profiles`。請務必選取 `baseline-profiles` 目錄。
3. 在 Android Studio 匯入專案時，請確定您可以執行 `app` 模組，建構之後要使用的範例應用程式。

範例應用程式

在本程式碼研究室中，您將使用 [JetSnack](#) 範例應用程式。這是一款使用 Jetpack Compose 編寫的虛擬點心訂購應用程式。

如要評估應用程式效能，您需要瞭解 UI 的結構及應用程式行為，才能透過基準測試存取 UI 元素。請執行應用程式並訂購點心，熟悉基本畫面內容。您不需要瞭解應用程式架構細節。



3. 什麼是基準設定檔

基準設定檔不須對內含的程式碼路徑進行解譯和**及時 (JIT)** 編譯步驟，因此首次啟動時的程式碼執行速度能加快約 30%。**Android 執行階段 (ART)** 會在應用程式或程式庫中傳送基準設定檔，可透過預先 (AOT) 編譯來最佳化其中的程式碼路徑，在每位新使用者加入和每次應用程式更新時提升效能。有了這項以設定檔為導引的最佳化功能 (PGO)，應用程式可最佳化啟動作業、減少互動卡頓情形，並從首次啟動就為使用者提升執行階段的整體效能。

導入基準設定檔後，所有使用者互動都會從首次執行起就更加順暢，無論是啟動應用程式、切換畫面，或是捲動內容都不例外。提升啟動和回應速度可增加每日活躍使用人數，還能提高平均回訪率。

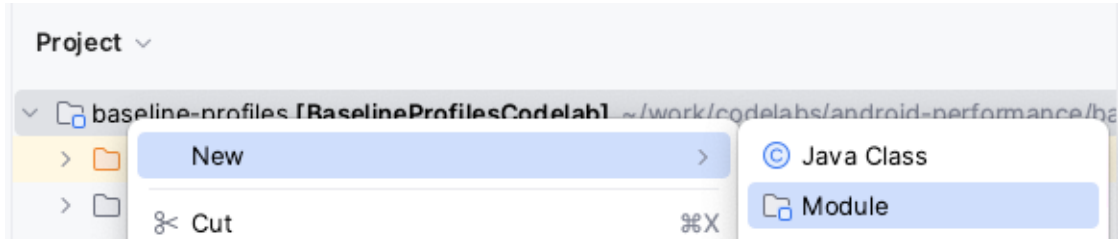
基準設定檔支援常見的使用者互動，可從首次啟動時就改善應用程式執行階段，且最佳化範圍不僅僅侷限於應用程式啟動步驟。引導式 AOT 編譯不需要使用者裝置，每發布一個版本即可在開發機器上 (而非行動裝置) 執行一次。發布帶有基準設定檔的版本時，應用程式最佳化作業的速度會比只依賴**雲端設定檔**時更快。

不使用基準設定檔時，所有應用程式程式碼都會在解譯後於記憶體中進行 JIT 編譯，或是在裝置處於閒置狀態時從背景寫入「odex」檔案。使用者在安裝或更新後第一次執行應用程式時，使用體驗可能會不理想，但一旦新的程式碼路徑經過最佳化，情況就會有所改善。

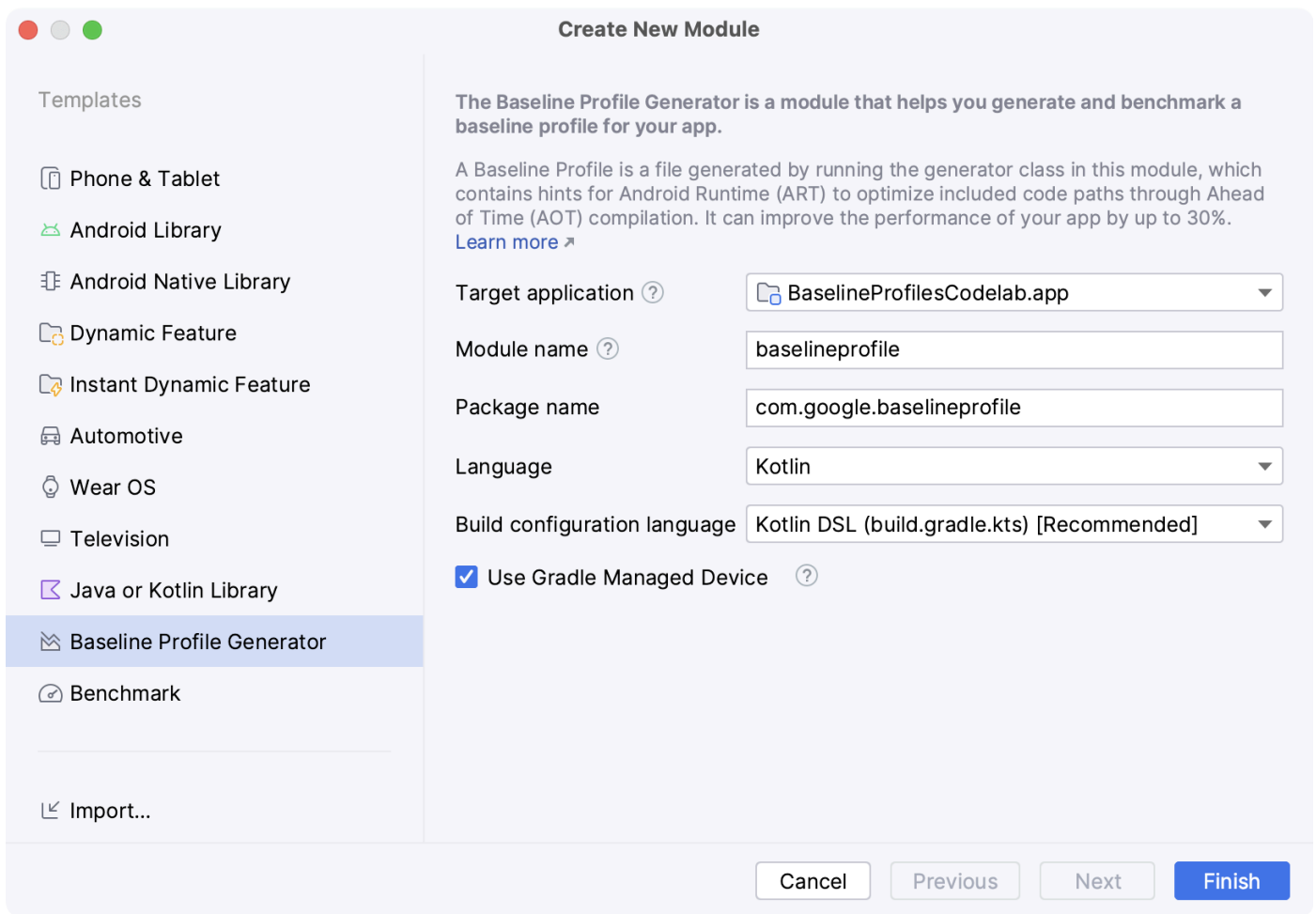
4. 設定基準設定檔產生器模組

您可以透過檢測設備測試類別產生基準設定檔。使用此類別時，必須在專案中加入新的 Gradle 模組。如要將模組加入專案，最簡單的方法是使用 Android Studio Hedgehog 以上版本內建的 Android Studio 模組精靈。

在「Project」面板中的專案或模組上按一下滑鼠右鍵，然後依序選取「New」>「Module」，開啟新的模組精靈視窗。



在開啟的視窗中，選取「Templates」窗格中的「Baseline Profile Generator」。



除了模組名稱、套件名稱、語言或建構設定語言等一般參數之外，新模組還有以下兩種不常用的輸入項目：**Target application** 和 **Use Gradle Managed Device**。

「Target application」是用來產生基準設定檔的應用程式模組。如果專案中有多個應用程式模組，請選取一個用來執行產生器的模組。

選取「Use Gradle Managed Device」核取方塊可設定模組，在自動管理的 Android 模擬器上執行基準設定檔產生器。如要進一步瞭解 Gradle 管理的裝置，請參閱「[使用 Gradle 管理的裝置擴大測試](#)」。如果取消勾選核取方塊，產生器會使用任何已連結的裝置。

注意：基準設定檔產生器須在已解鎖裝置或是模擬器上執行；如未取得 Root 權限，則須在 Android 13 (API 級別 33) 以上版本執行。

定義新模組的所有詳細資料後，請按一下「Finish」建立模組。

模組精靈變更的內容

模組精靈會變更專案的部分內容。

精靈會新增一個 Gradle 模組，名為 `baselineprofile` 或您在精靈內選擇的名稱。

這個模組會使用 `com.android.test` 外掛程式，要求 Gradle 不要將模組納入應用程式，因此模組能只包含測試程式碼或基準測試。此模組也會套用 `androidx.baselineprofile` 外掛程式，讓系統自動產生基準設定檔。

精靈也會變更您指定的目標應用程式模組。具體來說，它會套用 `androidx.baselineprofile` 外掛程式、新增 `androidx.profileinstaller` 依附元件，並將 `baselineProfile` 依附元件新增至新建立的模組 `build.gradle(.kts)`：

```
plugins {  
    id("androidx.baselineprofile")  
}  
  
dependencies {  
    // ...  
    implementation("androidx.profileinstaller:profileinstaller:1.3.0")  
    "baselineProfile"(project(mapOf("path" to ":baselineprofile")))  
}
```

新增 `androidx.profileinstaller` 依附元件後，就能執行下列操作：

- 在本機驗證所產生基準設定檔的效能提升幅度。
- 在不支援雲端設定檔的 Android 7 (API 級別 24) 及 Android 8 (API 級別 26) 上使用基準設定檔。
- 在沒有 Google Play 服務的裝置上使用基準設定檔。

`baselineProfile(project(":baselineprofile"))` 依附元件可告知 Gradle 要從哪個模組取得產生的基準設定檔。

您已完成專案設定，可以開始編寫基準設定檔產生器類別了。

5. 撰寫基準設定檔產生器

您通常需要為應用程式的一般使用者歷程產生基準設定檔。

模組精靈會建立基本的 `BaselineProfileGenerator` 測試類別，用來產生應用程式啟動作業的基準設定檔，如下所示：

```
@RunWith(AndroidJUnit4::class)
@LargeTest
class BaselineProfileGenerator {

    @get:Rule
    val rule = BaselineProfileRule()

    @Test
    fun generate() {
        rule.collect("com.example.baselineprofiles_codelab") {
            // This block defines the app's critical user journey. This is where you
            // optimize for app startup. You can also navigate and scroll
            // through your most important UI.

            // Start default activity for your app.
            pressHome()
            startActivityAndWait()

            // TODO Write more interactions to optimize advanced journeys of your app.
            // For example:
            // 1. Wait until the content is asynchronously loaded.
            // 2. Scroll the feed content.
            // 3. Navigate to detail screen.

            // Check UiAutomator documentation for more information about how to interact
            with the app.
            // https://d.android.com/training/testing/other-components/ui-automator
        }
    }
}
```

這個類別使用 `BaselineProfileRule` 測試規則，包含一個用來產生設定檔的測試方法。產生設定檔的進入點為 `collect()` 函式。這個函式只需要兩個參數：

- `packageName`：應用程式套件。
- `profileBlock`：最後一個 lambda 參數。

在 `profileBlock` lambda 中，您要指定涵蓋應用程式一般使用者歷程的互動方式。程式庫會多次執行 `profileBlock`、收集已呼叫的類別和函式，並在裝置上產生基準設定檔，其中包含要最佳化的程式碼。

根據預設，建立的產生器類別包含用來啟動預設 `Activity` 的互動，並且使用 `startActivityAndWait()` 方法，等到系統轉譯應用程式的第一個影格再開始互動。

運用自訂歷程擴充產生器

您可以發現產生的類別還包含一些 `TODO`，可編寫更多互動，最佳化應用程式的進階歷程。建議您採取這種做法，方便最佳化應用程式啟動後的效能。

在範例應用程式中，您可以按照下列步驟來辨識這些歷程：

1. 啟動應用程式。產生的類別已涵蓋部分內容。
2. 等待系統以非同步方式載入內容。
3. 捲動點心清單。
4. 前往點心詳細資料。

變更產生器，納入下方程式碼片段中涵蓋一般歷程的概略函式：

```
// ...
rule.collect("com.example.baselineprofiles_codelab") {
    // This block defines the app's critical user journey. This is where you
    // optimize for app startup. You can also navigate and scroll
    // through your most important UI.

    // Start default activity for your app.
    pressHome()
    startActivityAndWait()

    // TODO Write more interactions to optimize advanced journeys of your app.
    // For example:
    // 1. Wait until the content is asynchronously loaded.
    waitForAsyncContent()
    // 2. Scroll the feed content.
    scrollSnackListJourney()
    // 3. Navigate to detail screen.
    goToSnackDetailJourney()

    // Check UiAutomator documentation for more information about how to interact with the
    app.
    // https://d.android.com/training/testing/other-components/ui-automator
}
// ...
```

接著，為前面提到的每個歷程編寫互動。您可以撰寫成 `MacrobenchmarkScope` 的擴充功能函式，方便存取其中提供的參數和函式。用這種方式編寫，即可在基準測試中重複利用這些互動，驗證效能提升幅度。

等待非同步內容

許多應用程式會在啟動時以非同步方式載入內容 (又稱為**完整顯示狀態**)，告知系統何時要載入及轉譯內容，以及使用者何時能與應用程式互動。等候這些互動在產生器中顯示狀態 (`waitForAsyncContent`)：

1. 找出動態點心清單。
2. 等候清單中的某些項目顯示在畫面上。

```
fun MacrobenchmarkScope.waitForAsyncContent() {
    device.wait(Until.hasObject(By.res("snack_list")), 5_000)
    val contentList = device.findObject(By.res("snack_list"))
    // Wait until a snack collection item within the list is rendered.
    contentList.wait(Until.hasObject(By.res("snack_collection")), 5_000)
}
```

注意：如要最佳化在完整顯示狀態前觸發的所有程式碼，請在應用程式中呼叫 `ReportDrawn`，將事件發生時間告知系統。`androidx.activity` 程式庫中含有能輕鬆傳送信號的 [API](#)。

捲動清單歷程

您可以按照以下互動步驟操作，完成捲動點心清單歷程 (`scrollSnackListJourney`)：

1. 找出點心清單的 UI 元素。
2. 設定不會觸發系統操作機制的手勢邊界。
3. 捲動清單，並等到 UI 穩定為止。

```
fun MacrobenchmarkScope.scrollSnackListJourney() {
    val snackList = device.findObject(By.res("snack_list"))
    // Set gesture margin to avoid triggering gesture navigation.
    snackList.setGestureMargin(device.displayWidth / 5)
    snackList.fling(Direction.DOWN)
    device.waitForIdle()
}
```

前往詳細資料歷程

最後的歷程 (`goToSnackDetailJourney`) 會實作以下互動：

1. 找出可使用的點心清單和所有點心清單項目。
2. 選取清單內的項目。
3. 按一下該項目，並等到詳細資訊畫面載入為止。點心清單不會再出現在螢幕畫面上，您可以利用這一點。

```
fun MacrobenchmarkScope.goToSnackDetailJourney() {
    val snackList = device.findObject(By.res("snack_list"))
    val snacks = snackList.findObjects(By.res("snack_item"))
    // Select snack from the list based on running iteration.
    val index = (iteration ?: 0) % snacks.size
    snacks[index].click()
    // Wait until the screen is gone = the detail is shown.
    device.wait(Until.gone(By.res("snack_list")), 5_000)
}
```

定義執行基準設定檔產生器所需的所有互動後，您需要定義用來執行產生器的裝置。

6. 備妥用來執行產生器的裝置

注意：如果您已在步驟 3 於新模組精靈中勾選「Using Gradle Managed Device」，則系統已為您建立裝置定義，因此您可以略過這個步驟。

如要產生基準設定檔，建議您使用模擬器 (例如 Gradle 管理的裝置)，或搭載 Android 13 (API 33) 以上版本的裝置。

如要能重現過程並自動產生基準設定檔，可以使用 Gradle 管理的裝置。您可以利用 Gradle 管理的裝置在 Android 模擬器中執行測試，而且不需手動啟動及拆除模擬器。如要進一步瞭解 Gradle 管理的裝置，請參閱「[使用 Gradle 管理的裝置調整測試](#)」。

如要定義 Gradle 管理的裝置，請將定義新增至 `:baselineprofile` 模組 `build.gradle.kts` 檔案，如以下程式碼片段所示：

```
android {
    // ...

    testOptions.managedDevices.devices {
        create<ManagedVirtualDevice>("pixel6Api31") {
            device = "Pixel 6"
            apiLevel = 31
            systemImageSource = "aosp"
        }
    }
}
```

在本例中，我們使用 Android 11 (API 級別 31)，且 `aosp` 系統映像檔可以提供 Root 權限。

接下來，請設定基準設定檔 Gradle 外掛程式，這樣才能使用已定義的 Gradle 管理裝置。方法是在 `managedDevices` 屬性中加入裝置名稱，然後停用 `useConnectedDevices`，如以下程式碼片段所示：

```
android {
    // ...
}

baselineProfile {
    managedDevices += "pixel6Api31"
    useConnectedDevices = false
}

dependencies {
    // ...
}
```

注意：如要使用已連結的裝置產生基準設定檔，請在 `baselineProfile` 設定區塊中啟用 `useConnectedDevices` 參數。

接著，產生基準設定檔。

7. 產生基準設定檔

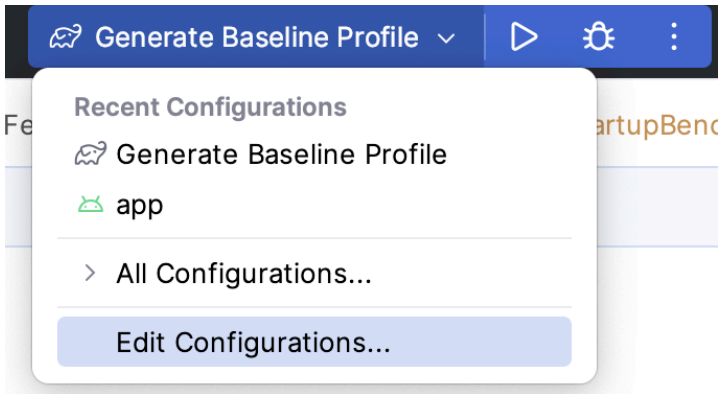
裝置準備就緒後，您就可以建立基準設定檔。基準設定檔 Gradle 外掛程式會建立 Gradle 工作，自動執行產生器測試類別的完整程序，並將產生的基準設定檔套用至您的應用程式。

新模組精靈會建立執行設定，方便快速執行 Gradle 工作，同時執行所有必要參數，不必在終端機和 Android Studio 之間切換。

請找到 `Generate Baseline Profile` 執行設定，然後點選「Run」按鈕



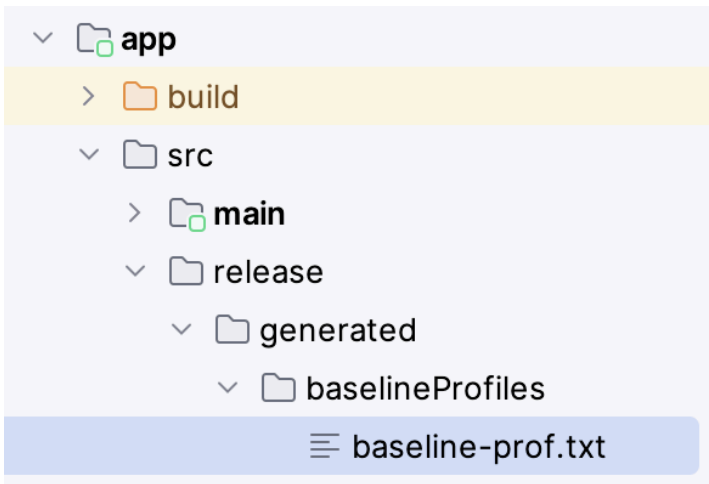
開始執行。



該工作會啟動先前定義的模擬器映像檔。多次執行 `BaselineProfileGenerator` 測試類別中的互動，然後拆除模擬器，並將輸出內容提供給 Android Studio。

警告：目前無法透過空白邊動作執行基準設定檔產生器。該測試可正常運作，但 Gradle 外掛程式不會將產生的檔案放入適當的資料夾。

產生器成功處理完畢後，Gradle 外掛程式會自動將產生的 `baseline-prof.txt` 放入 `src/release/generated/baselineProfile/` 資料夾中的目標應用程式 (`:app` 模組)。



(選用) 從指令列執行產生器

或者，您也可以從指令列執行產生器。您可以利用 Gradle 管理的裝置建立的工件：`:app:generateBaselineProfile`。這個指令會執行 `baselineProfile(project(:baselineProfile))` 依附元件所定義專案中的所有測試。由於此模組也包含之後用來驗證效能提升幅度的基準測試，因此這些測試會失敗，並顯示警告訊息，指出不應在模擬器上執行基準測試。

```
android
    .testInstrumentationRunnerArguments
    .androidx.benchmark.enabledRules=BaselineProfile
```

因此，您可以使用下列檢測執行器引數，篩選所有基準設定檔產生器，並略過所有基準測試：

完整指令如下所示：

```
./gradlew :app:generateBaselineProfile -
Pandroid.testInstrumentationRunnerArguments.androidx.benchmark.enabledRules=BaselineProfile
```

使用基準設定檔發布應用程式

產生基準設定檔並複製到應用程式原始碼後，請照常建構正式版應用程式。您不需額外將基準設定檔發布給使用者。Android Gradle 外掛程式會在建構期間挑選這些設定檔，納入 AAB 或 APK。接著，請將版本上傳至 Google Play。

使用者安裝應用程式或更新舊版應用程式時，也會一併安裝基準設定檔，進而提升應用程式首次執行時的效能。

下一步將說明如何驗證基準設定檔對應用程式效能提升幅度的影響。

8. (選用) 自訂基準設定檔的產生方式

基準設定檔 Gradle 外掛程式提供自訂選項，方便您依自身需求自訂設定檔的產生方式。您可以在建構指令碼中使用 `baselineProfile { }` 設定區塊來變更行為。

`:baselineprofile` 模組中的設定區塊會影響產生器的執行方式，可能會新增 `managedDevices`，並決定是否要新增到 `useConnectedDevices` 或 Gradle 管理的裝置。

`:app` 目標模組中的設定區塊會決定設定檔的儲存位置和產生方式。您可以變更下列參數：

- `automaticGenerationDuringBuild`：啟用後，即可在建構正式版的發布子版本時，產生基準設定檔。如想在提交應用程式前於 CI 上建構應用程式，這項功能就十分實用。
 - `saveInSrc`：指定產生的基準設定檔是否儲存在 `src/` 資料夾中。或者，您也可以從 `:baselineprofile` 建構資料夾中存取檔案。
 - `baselineProfileOutputDir`：定義產生的基準設定檔要儲存的位置。
 - `mergeIntoMain`：根據預設，系統會依據每個建構變化版本 (變種版本和建構類型) 產生基準設定檔。如要將所有設定檔合併為 `src/main`，請啟用此標記。
 - `filter`：您可以篩選要在產生的基準設定檔中，加入或排除哪些類別或方法。如果程式庫開發人員只想加入來自程式庫的程式碼，這項功能就十分實用。
-

9. 驗證啟動效能提升幅度

產生基準設定檔並將其新增至應用程式後，請驗證它是否對您應用程式的效能產生預期效果。

新的模組精靈會建立名為 `StartupBenchmarks` 的基準類別，其中包含用來測量應用程式啟動時間的基準測試，並與應用程式使用基準設定檔時的啟動時間比較。

該類別如下所示：

```
@RunWith(AndroidJUnit4::class)
@LargeTest
class StartupBenchmarks {

    @get:Rule
    val rule = MacrobenchmarkRule()

    @Test
    fun startupCompilationNone() =
        benchmark(CompilationMode.None())

    @Test
    fun startupCompilationBaselineProfiles() =
        benchmark(CompilationMode.Partial(BaselineProfileMode.Require))

    private fun benchmark(compilationMode: CompilationMode) {
        rule.measureRepeated(
            packageName = "com.example.baselineprofiles_codelab",
            metrics = listOf(StartupTimingMetric()),
            compilationMode = compilationMode,
            startupMode = StartupMode.COLD,
            iterations = 10,
            setupBlock = {
                pressHome()
            },
            measureBlock = {
                startActivityAndWait()

                // TODO Add interactions to wait for when your app is fully drawn.
                // The app is fully drawn when Activity.reportFullyDrawn is called.
                // For Jetpack Compose, you can use ReportDrawn, ReportDrawnWhen and
                ReportDrawnAfter
                // from the AndroidX Activity library.

                // Check the UiAutomator documentation for more information on how to
                // interact with the app.
                // https://d.android.com/training/testing/other-components/ui-automator
            }
        )
    }
}
```

該類別使用的 `MacrobenchmarkRule` 能為應用程式執行基準測試，並收集成效指標。編寫基準測試的進入點為規則中的 `measureRepeated` 函式。

此函式需要多個參數：

- `packageName`：要測量的應用程式。
- `metrics`：要在基準測試期間測量的[資訊類型](#)。
- `iterations`：重複執行基準測試的次數。
- `startupMode`：您希望應用程式在基準測試開始時的[啟動](#)方式。
- `setupBlock`：測量前必須發生的應用程式互動。
- `measureBlock`：要在基準測試期間測量的應用程式互動。

該測試類別也包含兩個測試：`startupCompilationNone()` 和 `startupCompilationBaselineProfiles()`，兩者會使用不同的 `compilationMode` 呼叫 `benchmark()` 函式。

CompilationMode

`CompilationMode` 參數會定義應用程式預先編譯至機器碼的方式。此參數提供以下選項：

- `DEFAULT`：在適用情況下，系統會使用基準設定檔預先編譯應用程式的部分內容。如未套用任何 `compilationMode` 參數，就會使用此選項。
- `None()`：重設應用程式編譯狀態，且不會預先編譯應用程式。應用程式執行作業期間依然會啟用[及時 \(JIT\) 編譯](#)。
- `Partial()`：使用基準設定檔和/或暖身執行作業，預先編譯應用程式。
- `Full()`：預先編譯全部應用程式程式碼。Android 6 (API 23) 以下版本只能使用此選項。

如果想開始最佳化應用程式效能，可以選擇 `DEFAULT` 編譯模式，因為此模式下應用程式的效能會與透過 Google Play 安裝時的效能類似。如果想比較基準設定檔帶來的效能提升幅度，您可以比較 `None` 和 `Partial` 這兩種編譯模式的結果。

注意：如要瞭解 `measureRepeated` 函式中使用的所有參數，請參閱「[編寫 Macrobenchmark](#)」，或是完成「[使用 Jetpack Macrobenchmark 檢查應用程式效能](#)」程式碼研究室。

修改基準測試以等待內容

基準測試與基準設定檔產生器的編寫方式類似，也就是編寫應用程式互動。根據預設，建立的基準測試只會等待第一個影格轉譯 (類似 `BaselineProfileGenerator` 的做法)，因此建議您改進基準測試，等待非同步內容。

如要這麼做，您可以重複使用為產生器編寫的擴充功能函式。由於這個基準測試會使用 `StartupTimingMetric()` 擷取啟動時間，建議您只在這裡加入等待非同步內容的動作，然後為產生器中定義的其他使用者歷程編寫不同的基準測試。

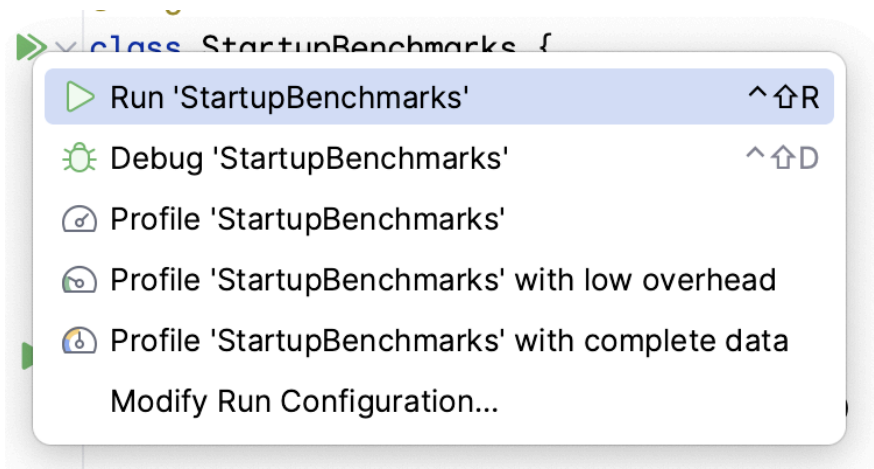
```
// ...
measureBlock = {
    startActivityAndWait()

    // The app is fully drawn when Activity.reportFullyDrawn is called.
    // For Jetpack Compose, you can use ReportDrawn, ReportDrawnWhen and ReportDrawnAfter
    // from the AndroidX Activity library.
    waitForAsyncContent() // <----- Added to wait for async content.

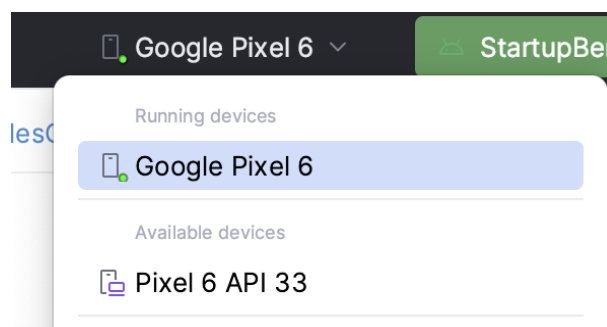
    // Check the UiAutomator documentation for more information on how to
    // interact with the app.
    // https://d.android.com/training/testing/other-components/ui-automator
}
```

執行基準測試

執行基準測試的方式與執行檢測設備測試的方式相同。您可以執行測試函式，或用旁邊的空白邊圖示執行整個類別。



請確定您已選定實體裝置，因為在 Android 模擬器上執行基準測試會在執行階段失敗，並顯示警告訊息，指出基準測試會提供錯誤結果。雖然嚴格來說，基準測試可以在模擬器上執行，但這麼做等同於評估主體機器的效能。如果主機負載量較大，基準測試的執行速度就會較慢，反之亦然。



執行基準測試後，系統會重構應用程式並執行您的基準測試。根據您定義的 `iterations`，基準測試會多次啟動、停止，甚至重新安裝應用程式。

基準測試完成後，您就能在 Android Studio 輸出內容中看到時間，如以下螢幕截圖所示：

Status2 passed2 tests, 1m 45 s 589 rTest Results

Filter tests:

✓

⊗

◇

×

↑

↓

>

Tests

Test Results

StartupBenchmarks

startupCompilationBaselineProfiles

startupCompilationNone

Starting 2 tests on Pixel 6 - 13
Connected to process 10172 on device 'google-pixel_6-19181FDF600D1J'.

StartupBenchmarks_startupCompilationBaselineProfiles
timeToFullDisplayMs [min 606.6](#), [median 637.9](#), [max 673.8](#)
timeToInitialDisplayMs [min 173.5](#), [median 193.7](#), [max 231.9](#)
Traces: Iteration [0](#) [1](#) [2](#) [3](#) [4](#) [5](#) [6](#) [7](#) [8](#) [9](#)
StartupBenchmarks_startupCompilationNone
timeToFullDisplayMs [min 808.5](#), [median 818.8](#), [max 847.4](#)
timeToInitialDisplayMs [min 195.6](#), [median 202.2](#), [max 212.1](#)
Traces: Iteration [0](#) [1](#) [2](#) [3](#) [4](#) [5](#) [6](#) [7](#) [8](#) [9](#)

BUILD SUCCESSFUL in 1m 45s
64 actionable tasks: 1 executed, 63 up-to-date

Build Analyzer results available

在螢幕截圖中，您可以看到每個 `CompilationMode` 都有不同的應用程式啟動時間，中位值如下表所示：

	timeToInitialDisplay [ms]	timeToFullDisplay [ms]
無	202.2	818.8
基準設定檔	193.7	637.9
提升幅度	4%	28%

`timeToFullDisplay` 編譯模式之間的差異為 **180 毫秒**，與只具備基準設定檔相比，效能大約提升 28%。`CompilationNone` 的效能較差，因為裝置在啟動應用程式時必須執行大部分的 JIT 編譯。`CompilationBaselineProfiles` 的效能較佳，因為使用基準設定檔 AOT 執行部分編譯時，會編譯使用者最可能使用的程式碼，而不預先編譯不重要的程式碼，讓系統不必立即載入這類程式碼。

注意：在正式版應用程式中，即使新增基準設定檔，效能提升效果仍可能較低。許多程式庫都隨附基準設定檔，因此應用程式效能已經過提升。

10. (選用) 驗證捲動效能提升幅度

與上一步驟類似，您可以測量並驗證捲動效能。首先，請建立包含基準測試規則的 `ScrollBenchmarks` 測試類別，以及使用不同編譯模式的兩種測試方法：

```
@LargeTest
@RunWith(AndroidJUnit4::class)
class ScrollBenchmarks {

    @get:Rule
    val rule = MacrobenchmarkRule()

    @Test
    fun scrollCompilationNone() = scroll(CompilationMode.None())

    @Test
    fun scrollCompilationBaselineProfiles() = scroll(CompilationMode.Partial())

    private fun scroll(compilationMode: CompilationMode) {
        // TODO implement
    }
}
```

在 `scroll` 方法中，使用含有必要參數的 `measureRepeated` 函式。如果是 `metrics` 參數，請使用 `FrameTimingMetric` 測量產生 UI 影格所需的時間：

```
private fun scroll(compilationMode: CompilationMode) {
    rule.measureRepeated(
        packageName = "com.example.baselineprofiles_codelab",
        metrics = listOf(FrameTimingMetric()),
        compilationMode = compilationMode,
        startupMode = StartupMode.WARM,
        iterations = 10,
        setupBlock = {
            // TODO implement
        },
        measureBlock = {
            // TODO implement
        }
    )
}
```

這次您需要在 `setupBlock` 和 `measureBlock` 之間進一步拆分互動，才能只測量第一個版面配置與捲動內容之間的影格持續時間。因此，請將啟動預設畫面的函式放入 `setupBlock`，並將已建立的擴充功能函式 `waitForAsyncContent()` 和 `scrollSnackListJourney()` 放入 `measureBlock`：

```
private fun scroll(compilationMode: CompilationMode) {
    rule.measureRepeated(
        packageName = "com.example.baselineprofiles_codelab",
        metrics = listOf(FrameTimingMetric()),
        compilationMode = compilationMode,
        startupMode = StartupMode.WARM,
        iterations = 10,
        setupBlock = {
            pressHome()
            startActivityAndWait()
        },
        measureBlock = {
            waitForAsyncContent()
            scrollSnackListJourney()
        }
    )
}
```

注意：請將 `waitForAsyncContent()` 移至 `setupBlock` 或 `measureBlock`，這取決於您想建立初始版面配置，還是只想捲動內容。

基準測試準備就緒後，您可以像先前一樣執行測試並取得結果，如以下螢幕截圖所示：

The screenshot shows the Android Studio Test Results interface. At the top, it indicates 'Status' as green, '2 passed', and '2 tests, 5 m 2 s 504 ms'. Below this, there are filter icons and a list of tests. The tests listed are 'Test Results', 'ScrollBenchmarks', 'scrollCompilationBaselineProfiles', and 'scrollCompilationNone', all marked with green checkmarks. The main area displays the results for 'ScrollBenchmarks_scrollCompilationBaselineProfiles' and 'ScrollBenchmarks_scrollCompilationNone'. For 'scrollCompilationBaselineProfiles', the frameDurationCpuMs values are P50: 4.4, P90: 18.4, P95: 20.2, P99: 27.0, and frameOverrunMs values are P50: -8.5, P90: 9.5, P95: 10.4, P99: 14.1. For 'scrollCompilationNone', the frameDurationCpuMs values are P50: 6.4, P90: 63.3, P95: 65.8, P99: 70.5, and frameOverrunMs values are P50: 38.6, P90: 54.4, P95: 59.3, P99: 79.4. Both tests show iteration traces from 0 to 9. At the bottom, it says 'BUILD SUCCESSFUL in 5m 2s'.

`FrameTimingMetric` 會在第 50、90、95 和第 99 個百分位數，以毫秒為單位輸出影格持續時間 (`frameDurationCpuMs`)。如果是 Android 12 (API 級別 31) 以上版本，則輸出結果也會傳回影格超出上限的時間 (`frameOverrunMs`)。這個值可以是負數，表示產生影格後還有多餘的時間。

您可以從結果發現 `CompilationBaselineProfiles` 的影格持續時間平均少了 2 毫秒，使用者可能不容易察覺這項差異。不過，其他百分位數的結果較明顯。P99 的差異為 **43.5 毫秒**，也就是在 90 FPS 的裝置上，系統略過的影格超過 3 個。舉例來說，Pixel 6 的影格轉譯時間上限為 1000 毫秒/90 FPS，一個影格不到 11 毫秒。

11. 恭喜

恭喜！您已成功完成本程式碼研究室，利用基準設定檔提升了應用程式效能！

其他資源

歡迎參考下列其他資源：

- [使用 Macrobenchmark 檢查應用程式效能](#)：此程式碼研究室會進一步探討基準測試。
- [效能範例](#)：此存放區含有 Macrobenchmark 和其他效能範例。
- [Now In Android 範例應用程式](#)：這是在實際工作環境運作的應用程式，會使用基準測試和基準設定檔提升效能。

參考文件

- <https://d.android.com/macrobenchmark>
 - <https://d.android.com/baseline-profiles>
-